Institut Polytechnique de Paris
ENSTA Paris

Projet Deep Learning II

Deep Neural Networks — RBM, DBN, DNN & VAE

Élaboré par :

WALHA Asma

Encadré par :

Yohan Petetin
Alasdair Newson

Mars 2026

Table des matières

1 Introduction **2**
1.1 Problématique . . . 2
1.2 Travail réalisé . . . 2
1.3 Organisation du rapport . . . 2

2 Étude RBM sur Binary AlphaDigits **3**
2.1 Présentation du modèle . . . 3
2.2 Données . . . 3
2.3 Résultats préliminaires . . . 3
2.4 Influence du nombre d'unités cachées . . . 4
2.5 Influence du nombre d'epochs . . . 5
2.6 Influence du nombre de classes . . . 6

3 Étude DBN sur Binary AlphaDigits **7**
3.1 Architecture du DBN . . . 7
3.2 Résultats préliminaires . . . 7
3.3 Influence de la profondeur . . . 7
3.4 Comparaison RBM vs DBN . . . 8

4 Étude DNN sur MNIST **9**
4.1 Protocole expérimental . . . 9
4.2 Figure 1 — Influence du nombre de couches . . . 9
4.3 Figure 2 — Influence du nombre de neurones par couche . . . 10
4.4 Figure 3 — Influence de la taille des données d'entraînement . . . 11
4.5 Meilleure configuration . . . 13

5 Bonus — Comparaison de modèles génératifs sur MNIST **14**
5.1 Motivations et choix du VAE . . . 14
5.2 Architecture du VAE . . . 14
5.3 Courbe d'apprentissage du VAE . . . 15
5.4 Comparaison visuelle . . . 15
5.5 Interpolation dans l'espace latent du VAE . . . 16
5.6 Conclusion . . . 16

6 Conclusion **17**
6.1 Bilan des résultats . . . 17
6.2 Limites observées . . . 17
6.3 Perspective historique . . . 17

7 Exécution du code **18**
7.1 Prérequis . . . 18
7.2 Scripts disponibles . . . 18
7.3 Exécution locale . . . 18

1 Introduction

Ce rapport présente le travail réalisé dans le cadre du Projet Deep Learning II. L'objectif principal est d'implémenter et d'analyser des architectures de réseaux de neurones profonds pré-entraînés de manière non supervisée, et d'évaluer leur apport par rapport à une initialisation aléatoire pour la tâche de classification de chiffres manuscrits.

1.1 Problématique

L'entraînement de réseaux de neurones profonds par rétropropagation seule pose un problème fondamental : plus le réseau est profond, plus les gradients tendent à s'annuler en remontant les couches (phénomène dit de *vanishing gradient*). Cette instabilité rend l'apprentissage difficile voire impossible pour des architectures à 4 couches ou plus avec une simple initialisation aléatoire.

Une approche proposée par Hinton et al. (2006) consiste à pré-entraîner chaque couche de manière non supervisée sous forme de *Restricted Boltzmann Machine* (RBM) avant d'affiner l'ensemble par rétropropagation. Ce pré-entraînement fournit un point de départ favorable dans l'espace des paramètres, permettant au gradient de converger plus efficacement.

1.2 Travail réalisé

Ce projet couvre l'ensemble de la chaîne suivante :

- **RBM** : implémentation de l'algorithme Contrastive Divergence-1 (CD-1), entraînement et génération d'images sur la base Binary AlphaDigits.
- **DBN** : empilement de RBMs entraînés couche par couche (greedy layer-wise pretraining).
- **DNN** : réseau de classification pré-entraîné via DBN puis affiné par rétropropagation sur MNIST, comparé à un réseau initialisé aléatoirement.
- **Bonus** : comparaison visuelle entre RBM, DBN et VAE (Variational Autoencoder) sur MNIST.

1.3 Organisation du rapport

Le rapport est organisé comme suit. La section 2 présente l'étude des RBMs sur la base Binary AlphaDigits avec analyse des hyperparamètres. La section 3 fait de même pour les DBNs. La section 4 constitue l'étude principale : analyse comparative des DNN pré-entraînés et aléatoires sur MNIST. La section 5 présente la comparaison avec le VAE. La section 6 conclut ce rapport, et la section 7 détaille comment exécuter le code.

2 Étude RBM sur Binary AlphaDigits

2.1 Présentation du modèle

Un Restricted Boltzmann Machine (RBM) est un modèle probabiliste à deux couches — une couche visible $\mathbf{v} \in \{0,1\}^p$ et une couche cachée $\mathbf{h} \in \{0,1\}^q$ — sans connexions intra-couche. L'énergie du système est définie par :

$$E(\mathbf{v}, \mathbf{h}) = -\mathbf{a}^\top \mathbf{v} - \mathbf{b}^\top \mathbf{h} - \mathbf{v}^\top W \mathbf{h} \tag{1}$$

Les distributions conditionnelles sont factorisées :

$$P(h_j = 1 \mid \mathbf{v}) = \sigma\left(b_j + \sum_i W_{ij} v_i\right) \tag{2}$$

$$P(v_i = 1 \mid \mathbf{h}) = \sigma\left(a_i + \sum_j W_{ij} h_j\right) \tag{3}$$

L'entraînement est réalisé par l'algorithme **Contrastive Divergence-1 (CD-1)**, qui approxime le gradient de la log-vraisemblance en une seule étape de reconstruction. Les mises à jour des paramètres sont :

$$\Delta W = \eta \left(\mathbf{v}^{(0)} \mathbf{h}^{(0)\top} - \mathbf{v}^{(1)} \mathbf{h}^{(1)\top}\right) / m \tag{4}$$

$$\Delta \mathbf{a} = \eta \left(\mathbf{v}^{(0)} - \mathbf{v}^{(1)}\right) / m \tag{5}$$

$$\Delta \mathbf{b} = \eta \left(\mathbf{h}^{(0)} - \mathbf{h}^{(1)}\right) / m \tag{6}$$

où m est la taille du mini-batch, η le taux d'apprentissage, et $\mathbf{v}^{(1)}, \mathbf{h}^{(1)}$ les reconstructions après une étape de Gibbs.

2.2 Données

La base Binary AlphaDigits contient 36 classes (chiffres 0–9 et lettres A–Z), chacune composée de 39 images binaires de taille $20 \times 16 = 320$ pixels. Les expériences ont été réalisées sur des sous-ensembles de caractères de taille variable.

2.3 Résultats préliminaires

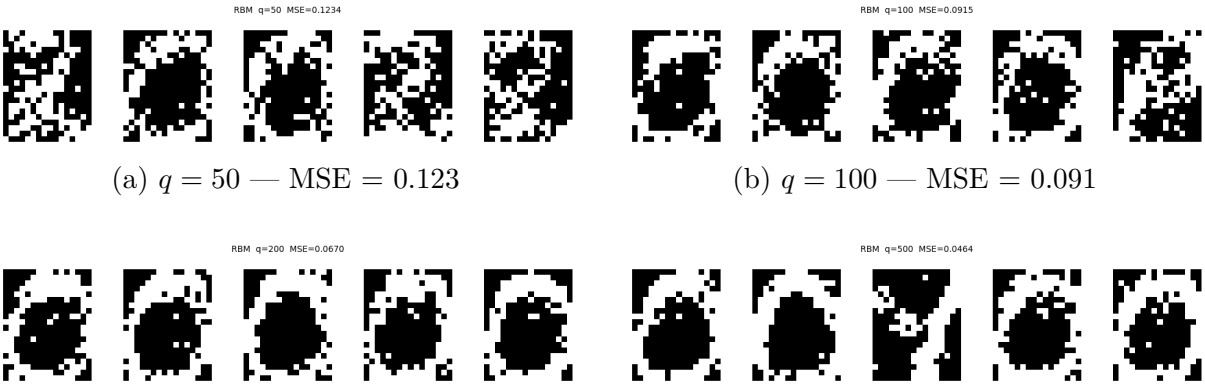
Un premier RBM avec 200 unités cachées, entraîné sur les caractères A, B et C (108 images) pendant 100 epochs (lr = 0.1, batch = 64), atteint une MSE de reconstruction de **0.067**. Les images générées par échantillonnage de Gibbs (500 itérations) présentent des formes reconnaissables mais bruitées, ce qui est attendu compte tenu du faible nombre d'échantillons d'entraînement.

FIGURE 1 – Images générées par un RBM (200 unités cachées, 100 epochs) entraîné sur les classes A, B, C.

2.4 Influence du nombre d'unités cachées

Nous avons entraîné des RBMs avec $q \in \{50, 100, 200, 500\}$ unités cachées (100 epochs, classes A/B/C).

FIGURE 2 – MSE de reconstruction en fonction du nombre d'unités cachées.



(a) $q = 50$ — MSE $= 0.123$ (b) $q = 100$ — MSE $= 0.091$

(c) $q = 200$ — MSE $= 0.067$ (d) $q = 500$ — MSE $= 0.047$

FIGURE 3 – Images générées selon le nombre d'unités cachées.

La MSE diminue de manière monotone avec q, illustrant que plus d'unités cachées permettent de capturer des dépendances plus complexes dans les données. On observe cependant des rendements décroissants : le gain de $q = 200$ à $q = 500$ est moins marqué que de $q = 50$ à $q = 200$. Visuellement, les images à $q = 500$ montrent des contours plus nets et une meilleure diversité.

2.5 Influence du nombre d'epochs

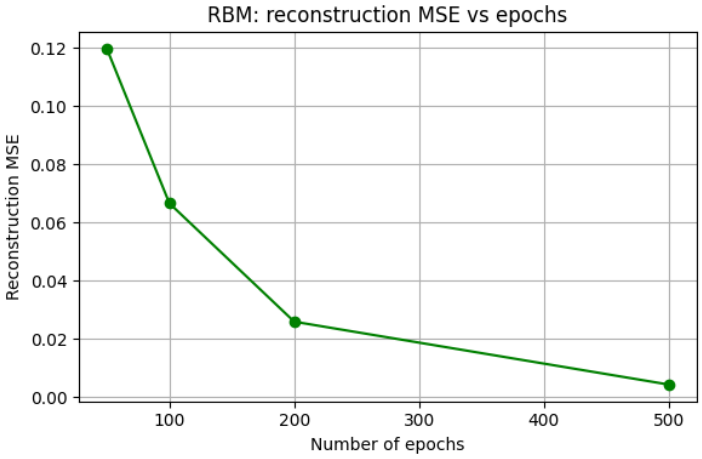


FIGURE 4 – MSE de reconstruction en fonction du nombre d'epochs (RBM, $q = 200$, classes A/B/C).

(a) 50 epochs — MSE = 0.120 (b) 500 epochs — MSE = 0.004

FIGURE 5 – Comparaison extrêmes : 50 vs 500 epochs.

L'augmentation du nombre d'epochs améliore significativement la qualité de reconstruction. La chute est la plus forte entre 50 et 200 epochs ; au-delà, la MSE se stabilise. Contrairement à l'apprentissage supervisé, l'entraînement prolongé d'un RBM ne conduit pas à du surapprentissage classique puisqu'il n'y a pas de labels — le modèle converge vers une meilleure approximation de la distribution des données.

2.6 Influence du nombre de classes

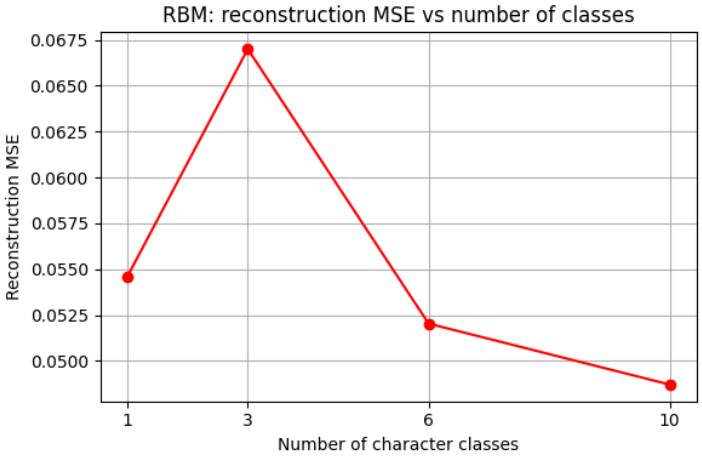


FIGURE 6 – MSE en fonction du nombre de classes appris ($q = 200$, 100 epochs).

La courbe présente un comportement non monotone : un pic à 3 classes, suivi d'une baisse à 6 et 10 classes. Cela résulte de deux effets antagonistes : (1) l'augmentation du nombre de classes rend la tâche plus difficile pour le modèle, et (2) elle fournit davantage de données d'entraînement (36 images par classe), ce qui stabilise les estimées CD-1. Au-delà de 3 classes, l'effet données domine, entraînant une baisse de la MSE. En revanche, visuellement, la qualité par classe se dégrade avec plus de classes car le modèle doit modéliser une distribution plus multimodale.

3 Étude DBN sur Binary AlphaDigits

3.1 Architecture du DBN

Un Deep Belief Network (DBN) est un empilement de RBMs. Son entraînement suit une procédure **greedy layer-wise** : chaque couche est entraînée successivement en utilisant comme entrée les représentations cachées de la couche précédente. Formellement, pour un DBN à L couches :

1. Entraîner le RBM_1 sur les données brutes $\mathbf{X}$.
2. Calculer $\mathbf{H}^{(1)} = P(\mathbf{h}^{(1)} = 1 \mid \mathbf{X})$ via $\sigma(W_1^\top \mathbf{X} + \mathbf{b}_1)$.
3. Entraîner le RBM_2 sur $\mathbf{H}^{(1)}$.
4. Répéter jusqu'à la dernière couche.

La génération d'images s'effectue en exécutant un échantillonneur de Gibbs sur la dernière couche RBM, puis en propageant les activations *en sens inverse* jusqu'à la couche visible.

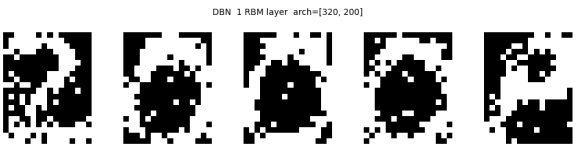
3.2 Résultats préliminaires

Un DBN [320 → 200 → 100] entraîné sur les classes A, B, C (100 epochs par couche) génère des images plus structurées que le RBM seul. La MSE de la première couche atteint 0.067, et la deuxième couche converge à 0.042.

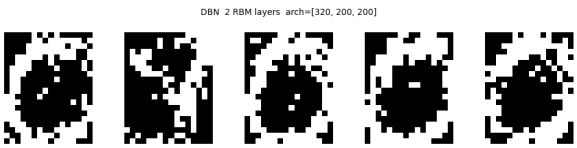
FIGURE 7 – Images générées par un DBN [320→200→100] entraîné sur les classes A, B, C.

3.3 Influence de la profondeur

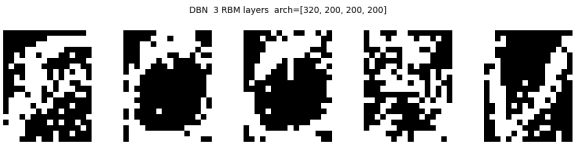
Nous avons comparé des DBNs de 1 à 4 couches RBM (200 unités par couche, classes A/B/C, 100 epochs).



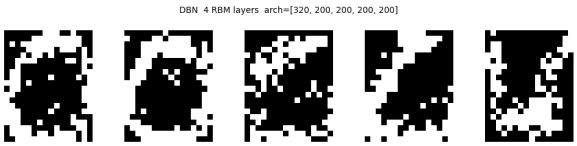
(a) 1 couche RBM



(b) 2 couches RBM



(c) 3 couches RBM



(d) 4 couches RBM

FIGURE 8 – Images générées par des DBNs de profondeur croissante (200 unités/couche, classes A/B/C).

On observe une progression de la qualité visuelle avec la profondeur. La couche supplémentaire de chaque DBN extrait des représentations de plus en plus abstraites des données, ce qui améliore la cohérence globale des images générées. Cependant, le gain marginal diminue après 2–3 couches : la complexité calculatoire augmente (chaque couche nécessite 100 epochs), tandis que l'amélioration visuelle devient moins perceptible.

Un phénomène notable est que la MSE de la couche supérieure *décroit davantage* quand la profondeur augmente (la couche 4 du DBN-4 atteint une MSE finale de 0.006), car les représentations intermédiaires deviennent progressivement plus régulières et plus faciles à modéliser.

3.4 Comparaison RBM vs DBN

Table 1 – Comparaison RBM (1 couche, $q = 200$) vs DBN (2 couches, 200 unités chacune) sur A/B/C.

Modèle	**Paramètres**	**Qualité visuelle**
RBM ($q = 200$)	$320 \times 200 + 520$	Formes bruitées
DBN (2 couches)	$320 \times 200 + 200 \times 200 + \dots$	Formes plus cohérentes

Le DBN améliore la génération car chaque couche apprend à représenter des abstractions de niveau supérieur. Le premier RBM capture les corrélations locales de pixels ; le second capture des structures globales (formes des lettres). Cette hiérarchie de représentations est le principe fondateur des réseaux profonds.

4 Étude DNN sur MNIST

4.1 Protocole expérimental

Données : La base MNIST contient 60 000 images d'entraînement et 10 000 images de test de chiffres manuscrits (28×28 pixels, niveaux de gris). Les images sont binarisées avec un seuil de 127 : les pixels > 127 deviennent 1, les autres 0. Les labels sont encodés en one-hot (vecteurs de dimension 10).

Architecture : Un DNN est représenté comme un empilement de RBMs auquel on ajoute une couche de classification softmax. Pour une architecture $[784, n_1, \ldots, n_L, 10]$, les L premières couches sont des RBMs et la dernière est la couche softmax.

Comparaison : Pour chaque configuration, deux réseaux identiques sont initialisés :

- **DNN pré-entraîné** : les couches cachées sont pré-entraînées couche par couche via CD-1 (100 epochs), puis l'ensemble est affiné par rétropropagation (200 epochs).
- **DNN aléatoire** : les poids sont initialisés aléatoirement (N(0, 0.01)), puis directement entraîné par rétropropagation (200 epochs).

Paramètres fixes : Learning rate $\eta = 0.1$, mini-batch de taille 128, optimisation par descente de gradient stochastique.

Métrique : Le taux d'erreur est défini comme la proportion de prédictions incorrectes sur le jeu de test.

4.2 Figure 1 — Influence du nombre de couches

Configuration : 200 neurones par couche, 2 à 5 couches cachées, 60 000 données d'entraînement.

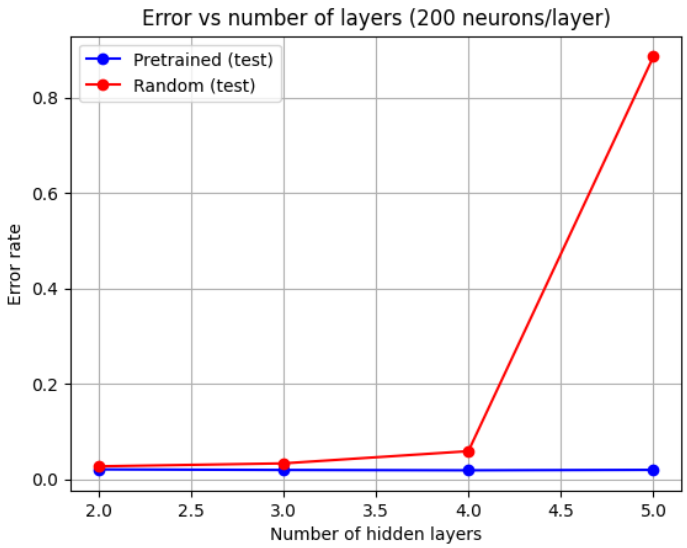
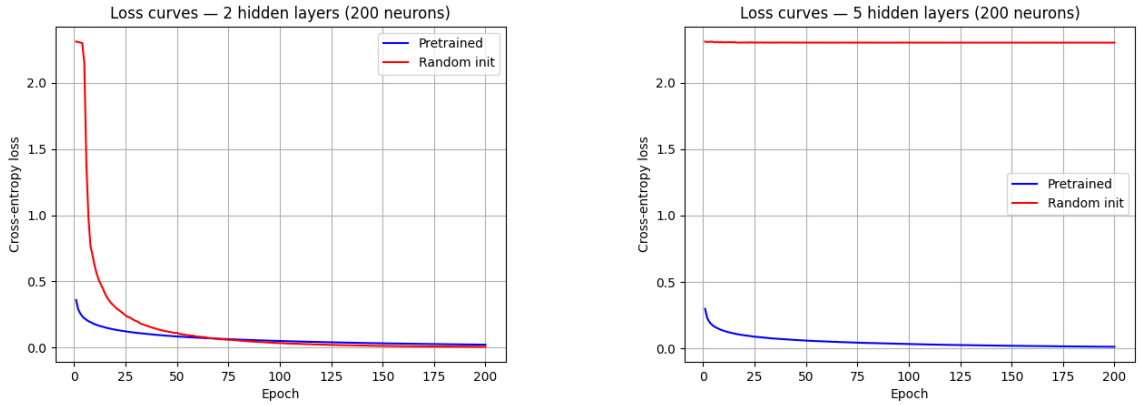


FIGURE 9 – Taux d'erreur test en fonction du nombre de couches cachées (200 neurones/-couche).



(a) Courbes de loss — 2 couches

(b) Courbes de loss — 5 couches

FIGURE 10 – Évolution de l'entropie croisée durant la rétropropagation pour 2 et 5 couches cachées.

Les résultats révèlent un contraste saisissant entre les deux réseaux :

— **DNN pré-entraîné** : le taux d'erreur reste stable autour de 2% quelle que soit la profondeur. Le pré-entraînement initialise les poids dans une région favorable de l'espace des paramètres, permettant au gradient de se propager efficacement même à travers 5 couches.

— **DNN aléatoire** : le taux d'erreur explose au-delà de 3 couches, atteignant **88%** à 5 couches — soit une performance quasi-aléatoire (10% théorique). Ceci illustre le problème classique du *vanishing gradient* : les gradients s'annulent en remontant les couches, empêchant toute mise à jour significative des premières couches.

Ce résultat confirme l'observation historique de Hinton et al. (2006) : le pré-entraînement non supervisé résout le problème du vanishing gradient en fournissant une initialisation proche d'un minimum acceptable.

4.3 Figure 2 — Influence du nombre de neurones par couche

Configuration : 2 couches cachées, $n \in \{100, 200, 300, 500, 700\}$ neurones par couche, 60 000 données.

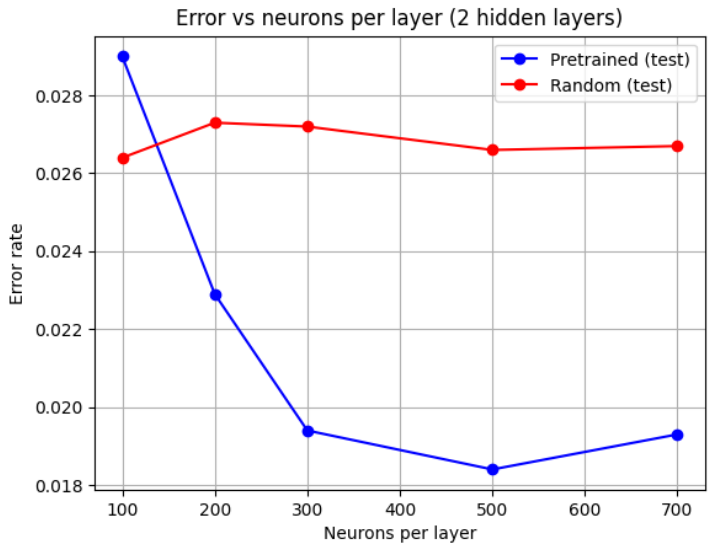


FIGURE 11 – Taux d'erreur test en fonction du nombre de neurones par couche (2 couches cachées).

Les deux réseaux s'améliorent avec l'augmentation du nombre de neurones, mais selon des dynamiques différentes :

— Le **DNN pré-entraîné** bénéficie davantage de la capacité accrue : la courbe descend plus steeply, atteignant ≈**1.9%** à 500 neurones. L'écart avec le réseau aléatoire est constant sur toute la plage.

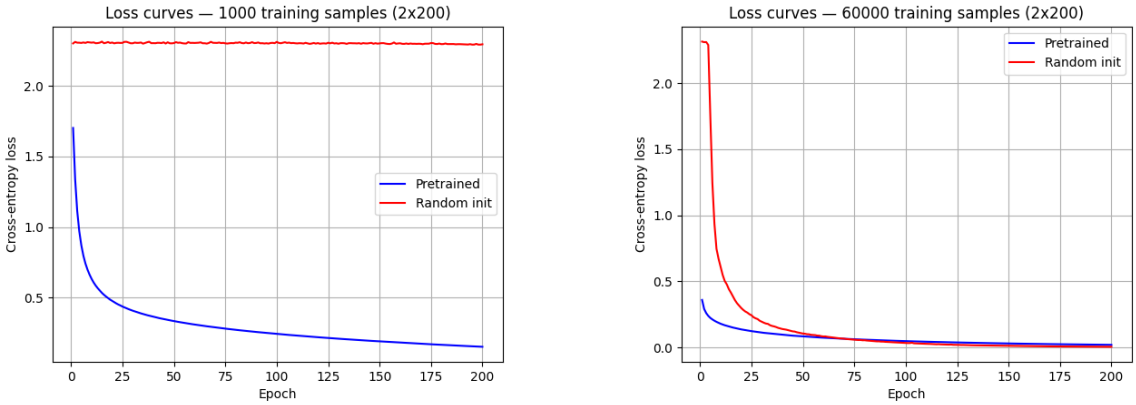
— Le **DNN aléatoire** améliore également ses performances, convergant vers ≈2.6% à 700 neurones. Avec seulement 2 couches, le problème de vanishing gradient est moins sévère, expliquant que même une initialisation aléatoire converge correctement.

La configuration optimale identifiée est **2 couches cachées × 500 neurones, pré-entraîné**, atteignant un taux d'erreur de ≈1.9%.

4.4 Figure 3 — Influence de la taille des données d'entraînement

Configuration : 2 couches de 200 neurones, $n_{\text{train}} \in \{1\,000, 3\,000, 7\,000, 10\,000, 30\,000, 60\,000\}$.

FIGURE 12 – Taux d'erreur test en fonction du nombre de données d'entraînement (2 couches × 200 neurones).



(a) Loss curves — 1 000 samples

(b) Loss curves — 60 000 samples

FIGURE 13 – Évolution de l'entropie croisée à 1 000 et 60 000 données d'entraînement.

C'est la figure la plus révélatrice concernant l'apport du pré-entraînement :

— Avec seulement **1 000 samples**, le DNN pré-entraîné atteint 11% d'erreur tandis que le réseau aléatoire dépasse 90%. L'écart est énorme : le pré-entraînement compense le manque de données labellisées en exploitant la structure non supervisée des données.

— L'écart se résorbe progressivement à mesure que les données augmentent. À **60 000 samples**, les deux réseaux convergent vers ≈2–3%, la quantité de données suffisant à guider la rétropropagation même sans pré-entraînement.

Le pré-entraînement agit comme une forme de **régularisation implicite** : il contraint l'initialisation à capturer la structure des données, réduisant ainsi la dépendance aux données labellisées. Cet effet est particulièrement précieux dans les régimes de faibles données, où l'annotation est coûteuse.

4.5 Meilleure configuration

En cherchant à minimiser le taux d'erreur test, la configuration optimale identifiée est :

Table 2 – Meilleure configuration DNN sur MNIST.

Paramètre	**Valeur**
Architecture	[784, 500, 500, 10]
Pré-entraînement	Oui (CD-1, 100 epochs/couche)
Rétropropagation	200 epochs
Learning rate	0.1
Mini-batch	128
Données d'entraînement	60 000
Taux d'erreur test	$\approx$ **1.9**%

5 Bonus — Comparaison de modèles génératifs sur MNIST

5.1 Motivations et choix du VAE

Le pré-entraînement par RBM et DBN repose sur la capacité des modèles à générer des données réalistes. Il est donc naturel de comparer ces modèles génératifs à des architectures plus récentes. Nous avons choisi le **Variational Autoencoder (VAE)** pour les raisons suivantes :

- Le VAE est un modèle à variables latentes, comme le RBM et le DBN, ce qui rend la comparaison théoriquement pertinente.
- Contrairement aux GANs, l'entraînement du VAE est stable et déterministe, garantissant des résultats reproductibles.
- Le VAE offre un espace latent **continu et régularisé**, permettant une interpolation entre les données générées — une capacité absente des RBMs.

5.2 Architecture du VAE

Le VAE est composé d'un encodeur et d'un décodeur :

$$\text{Encodeur} : \mathbf{x} \to (\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2) \in \mathbb{R}^d \times \mathbb{R}^d \tag{7}$$

$$\text{Reparamétrisation} : \mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, I) \tag{8}$$

$$\text{Décodeur} : \mathbf{z} \to \hat{\mathbf{x}} \in [0,1]^{784} \tag{9}$$

La fonction de perte est l'ELBO (Evidence Lower BOund) :

$$\mathcal{L} = \underbrace{-\mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x}|\mathbf{z})]}_{\text{reconstruction (BCE)}} + \underbrace{D_{\mathrm{KL}}(q(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))}_{\text{régularisation}} \tag{10}$$

Le terme KL contraint la distribution latente à rester proche d'une gaussienne standard $\mathcal{N}(\mathbf{0}, I)$, assurant que l'espace latent est régulier et continu.

Architecture utilisée : encodeur $[784 \to 256 \to 256 \to 64]$, décodeur $[64 \to 256 \to 256 \to 784]$, dimension latente $d = 64$, optimiseur Adam ($\eta = 0.001$), 30 epochs sur les 60 000 images MNIST. Total : 583 568 paramètres.

5.3 Courbe d'apprentissage du VAE

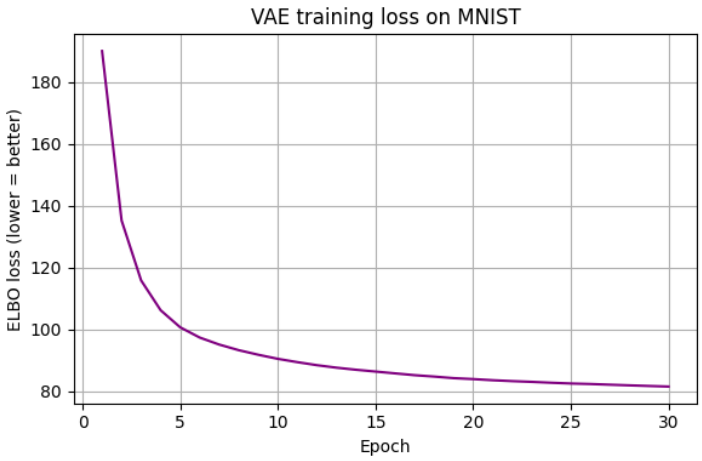


FIGURE 14 – Évolution de la loss ELBO durant l'entraînement du VAE (30 epochs, MNIST complet).

La loss passe de 190 à 81.5 en 30 epochs, avec une convergence rapide dans les 10 premières epochs puis un plateau. Le modèle est bien entraîné.

5.4 Comparaison visuelle

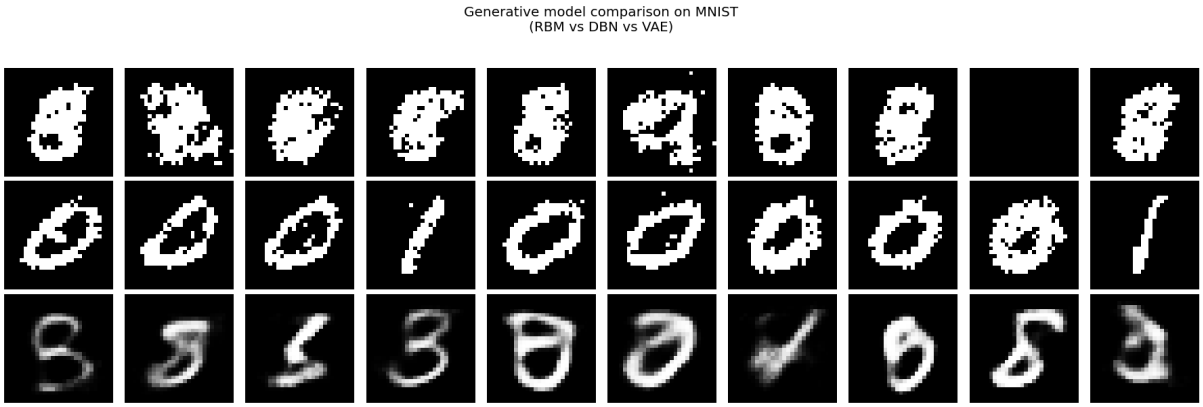


FIGURE 15 – Comparaison des images générées par RBM (ligne 1), DBN (ligne 2) et VAE (ligne 3) sur MNIST.

(a) RBM (b) DBN (2 couches) (c) VAE

FIGURE 16 – Images générées individuellement par chaque modèle.

Analyse qualitative.

— **RBM** : génère des formes binaires reconnaissables mais bruitées. L'échantillonnage de Gibbs dans l'espace binaire produit des images « éparpillées », sans lissage. La diversité est limitée car le Gibbs tend à rester dans les modes dominants.

— **DBN** (2 couches) : amélioration notable — les formes sont plus structurées et cohérentes. La hiérarchie de représentations permet de capturer des patterns de plus haut niveau (topologie des chiffres). Reste cependant binaire et bruité.

— **VAE** : qualité nettement supérieure. Les chiffres générés sont lisses, variés et clairement reconnaissables (2, 3, 8, 0...). L'espace latent continu permet de générer des variantes plus naturelles. La régularisation KL garantit une couverture uniforme de l'espace des données.

5.5 Interpolation dans l'espace latent du VAE

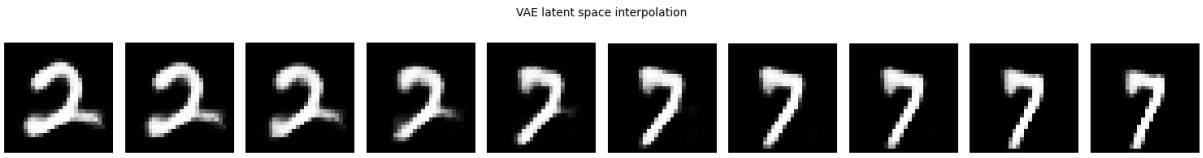


FIGURE 17 – Interpolation linéaire dans l'espace latent du VAE entre deux points aléatoires (transition 2 → 7).

La capacité d'interpolation du VAE est une démonstration directe que son espace latent est **sémantiquement structuré** : les points intermédiaires correspondent à des digits réalistes qui morphent progressivement d'un chiffre à un autre. Ni le RBM ni le DBN ne permettent ce type d'interpolation, car leurs espaces latents sont discrets et non régularisés.

5.6 Conclusion

La supériorité visuelle du VAE provient de son espace latent continu et de sa loss de reconstruction différentiable. Cependant, RBM et DBN conservent l'avantage d'une interprétabilité probabiliste directe et d'une complexité algorithmique plus faible — ce qui justifie leur utilisation comme étape de pré-entraînement dans les DNNs.

6 Conclusion

Ce projet a permis d'implémenter et d'analyser en profondeur la chaîne complète RBM → DBN → DNN pré-entraîné, en comparaison avec un DNN initialisé aléatoirement, sur la tâche de classification des chiffres manuscrits MNIST.

6.1 Bilan des résultats

Les trois figures principales de l'étude MNIST mettent en évidence des conclusions claires :

1. **Profondeur** : le pré-entraînement est *indispensable* pour les réseaux profonds. Au-delà de 3 couches, un réseau aléatoire ne converge plus (88% d'erreur à 5 couches), tandis que le réseau pré-entraîné reste stable à 2%.
2. **Largeur** : les deux architectures bénéficient de plus de neurones, mais le réseau pré-entraîné maintient un avantage constant. La configuration optimale identifiée est 2 couches × 500 neurones pré-entraîné (≈1.9% d'erreur).
3. **Données** : le pré-entraînement agit comme une régularisation implicite, particulièrement bénéfique en faible régime de données (90% d'erreur pour le réseau aléatoire contre 11% pour le pré-entraîné à 1 000 samples).

6.2 Limites observées

— L'algorithme CD-1 est une approximation du gradient de la log-vraisemblance. Des variantes comme CD-k avec $k > 1$ ou Persistent Contrastive Divergence (PCD) améliorent la qualité du modèle au prix d'un coût computationnel accru.

— Le RBM et le DBN génèrent des images binaires, ce qui les limite aux données binarisées. Ils ne peuvent pas modéliser de distributions continues sans modification architecturale.

— Le pré-entraînement perd son avantage avec suffisamment de données (60 000 samples), ce qui explique pourquoi il a été progressivement remplacé par des techniques de régularisation modernes (Dropout, BatchNorm, Adam).

6.3 Perspective historique

Ce projet illustre une étape clé de l'histoire du deep learning : le pré-entraînement non supervisé de Hinton et al. (2006) a permis d'entraîner les premiers réseaux profonds à une époque où les techniques de régularisation modernes n'existaient pas encore. Si aujourd'hui ces méthodes ont été largement supplantées par des architectures comme les Transformers, comprendre leur fonctionnement reste fondamental pour appréhender les principes de l'apprentissage de représentations.

La comparaison avec le VAE illustre l'évolution des modèles génératifs : d'un modèle probabiliste discret à espace latent non structuré (RBM) vers un modèle différentiable à espace latent continu et régularisé (VAE), permettant interpolation et génération de meilleure qualité.

7 Exécution du code

7.1 Prérequis

Environnement Python.

```
# Creer un environnement virtuel
python3 -m venv venv
source venv/bin/activate

# Installer les dependances
pip install -r requirements.txt
```

Le fichier `requirements.txt` contient :

```
torch>=2.0.0
numpy>=1.24.0
scipy>=1.10.0
matplotlib>=3.7.0
```

Données. Placer les fichiers de données dans les répertoires suivants :

- `data/binaryalphadigs.mat` — Binary AlphaDigits
- `data/minst/` — les 4 fichiers MNIST décompressés : `train-images-idx3-ubyte`, `train-labels-idx1-ubyte`, `t10k-images-idx3-ubyte`, `t10k-labels-idx1-ubyte`

7.2 Scripts disponibles

Table 3 – Scripts et leur utilisation.

Script	Description
`principal_RBM_alpha.py`	Entraînement RBM sur AlphaDigits, génération d'images
`principal_DBN_alpha.py`	Entraînement DBN sur AlphaDigits, génération d'images
`principal_alpha_study.py`	Étude complète des hyperparamètres AlphaDigits
`principal_DNN_MNIST.py`	Étude DNN pré-entraîné vs aléatoire (Fig 1, 2, 3)
`principal_bonus_VAE.py`	Comparaison RBM / DBN / VAE sur MNIST

7.3 Exécution locale

```
# Verification rapide sur AlphaDigits
python principal_RBM_alpha.py
python principal_DBN_alpha.py

# Etude hyperparametres AlphaDigits
python principal_alpha_study.py

# Etude principale MNIST (long)
```

```
python principal_DNN_MNIST.py

# Bonus VAE
python principal_bonus_VAE.py
```

Les figures sont sauvegardées dans `outputs/` (créé automatiquement).